Exercise 5 – Feature Effects

Interpretable Machine Learning

Hint: Useful libraries

R

```
library(ggplot2)
library(randomForest)
library(reshape2)
```

Python

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import statsmodels.api as sm
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import ParameterGrid
```

Exercise 1: PDP and ICE in case of interactions

Learning goals

1. Derive PD and ICE functions analytically for a model with an interaction term.
2. Explain why PDPs can obfuscate heterogeneous effects and why ICE curves should be displayed alongside them.
3. Implement ICE, PDP and their centered variants from scratch.

Let us assume that we fitted the following linear regression model with two features:

$$\hat{f}(\mathbf{x}) = \hat{f}(x_1, x_2) = \hat{\beta}_1 x_1 + \hat{\beta}_2 x_2 + \hat{\beta}_3 x_1 x_2 + \hat{\beta}_0.$$

a) Analytically derive the PD function of feature $S = \{1\}$ (with $C = \{2\}$).

Hint

In the lecture slides we derived the PD function for a linear regression model without an interaction term. In the end, your solution should include terms like the expected value of X_2.

Solution

Derivation of the PD function for $S = \{1\}$ (with $C = \{2\}$) given the model above:

$$\begin{aligned} f_{1,PD}(x_1) = \mathbb{E}_{x_2}\left(\hat{f}(x_1, x_2)\right) &= \int_{-\infty}^{\infty} \left(\hat{\beta}_1 x_1 + \hat{\beta}_2 x_2 + \hat{\beta}_3 x_1 x_2 + \hat{\beta}_0\right) d\mathbb{P}(x_2) \\ &= \hat{\beta}_1 x_1 + \int_{-\infty}^{\infty} \hat{\beta}_2 x_2 + \hat{\beta}_3 x_1 x_2 \, d\mathbb{P}(x_2) + \hat{\beta}_0 \\ &= \hat{\beta}_1 x_1 + \int_{-\infty}^{\infty} (\hat{\beta}_2 + \hat{\beta}_3 x_1) x_2 \, d\mathbb{P}(x_2) + \hat{\beta}_0 \\ &= \hat{\beta}_1 x_1 + (\hat{\beta}_2 + \hat{\beta}_3 x_1) \cdot \int_{-\infty}^{\infty} x_2 \, d\mathbb{P}(x_2) + \hat{\beta}_0 \\ &= \hat{\beta}_1 x_1 + (\hat{\beta}_2 + \hat{\beta}_3 x_1) \cdot \mathbb{E}_{x_2}(x_2) + \hat{\beta}_0 \end{aligned}$$

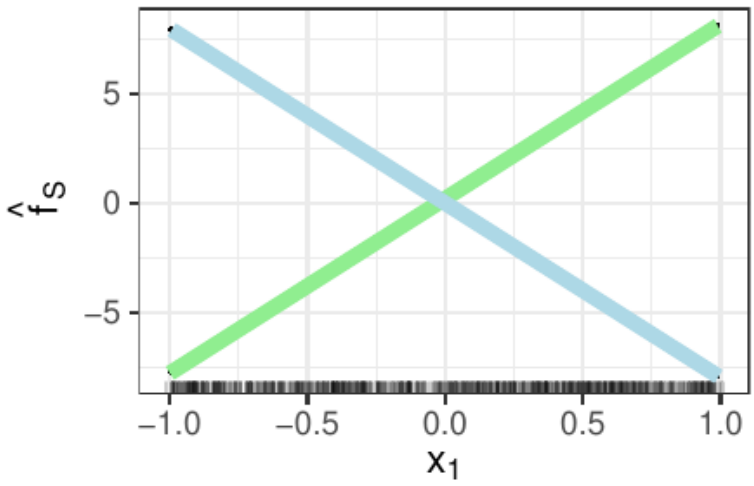
b) Let us assume that the estimated coefficients in the model above are $\hat{\beta}_0 = 0$, $\hat{\beta}_1 = -8$, $\hat{\beta}_2 = 0.2$ and $\hat{\beta}_3 = 16$. Furthermore, $X_1 \sim Unif(-1, 1)$ is uniformly distributed between -1 and 1, and $X_2 \sim B(1, 0.5)$ origins from a Bernoulli distribution. Compute the exact PD function of feature X_1 using your derived function of a).

Solution

PD function for $\hat{\beta}_0 = 0$, $\hat{\beta}_1 = -8$, $\hat{\beta}_2 = 0.2$, $\hat{\beta}_3 = 16$, $X_1 \sim Unif(-1, 1)$ and $X_2 \sim B(1, 0.5)$:

$$\begin{aligned} f_{1,PD}(x_1) = \hat{\beta}_1 x_1 + (\hat{\beta}_2 + 16x_1) \cdot \mathbb{E}_{x_2}(x_2) + \hat{\beta}_0 &= -8x_1 + (0.2 + 16x_1) \cdot \mathbb{E}_{x_2}(x_2) + 0 \\ &= -8x_1 + (0.2 + 16x_1) \cdot 0.5 \\ &= -8x_1 + 0.1 + 8x_1 \\ &= 0.1 \end{aligned}$$

c) The following plot displays the ICE curves of X_1. The rugs show the marginal distribution of X_1. Please note that since X_2 is binary, we do not receive n individual ICE curves, but indeed only two unique ones. Derive the conditional expectation functions $\hat{f}_1^{(i)}(x_1)$ of X_1 for group $X_2 = 1$ and for group $X_2 = 0$ using the model above given the estimated coefficients of b). Which color coding (light green or light blue) reflects the group $X_2 = 1$, which one group $X_2 = 0$?



Solution

ICE functions for group $X_2 = 1$ and for group $X_2 = 0$:

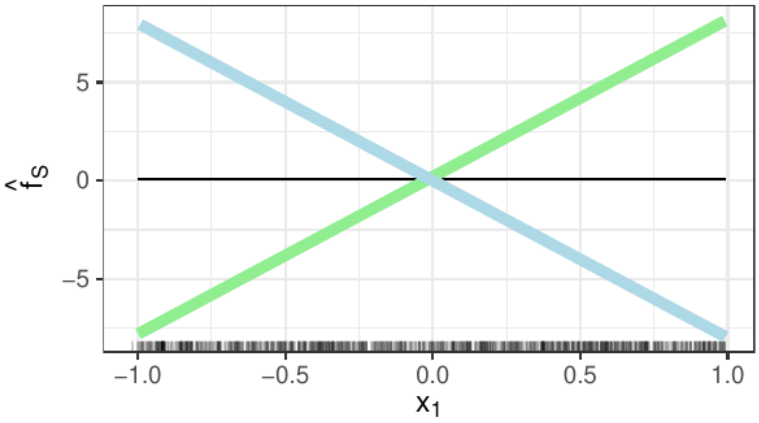
$$f_1(x_1) = \begin{cases} -8x_1 + (0.2 + 16x_1) \cdot 1 = 8x_1 + 0.2 & x_2 = 1 \\ -8x_1 + (0.2 + 16x_1) \cdot 0 = -8x_1 & x_2 = 0 \end{cases}$$

The light green dots correspond to group $X_2 = 1$, the light blue dots to group $X_2 = 0$.

d) Add the PDP you derived in b) to the plot. Use this example to explain briefly why it is advisable to display not only the PDP but also the ICE curves, when visualizing the feature effects of a specific model.

Solution

The example illustrates that by the averaging of ICE curves for a PD plot we might obfuscate heterogeneous effects and interactions. Although the ICE curves here show a strong effect of X_1 on Y, the effect is not apparent in the PDP. Therefore, it is highly recommended to plot PD plots and ICE curves together.



e) **Implementation: ICE and PDP.** Write functions that, given a fitted model object, a data matrix $\mathbf{X}$, a feature index j and a specified grid of values, do the following:

(i) `ice(model, X, j, grid)` – returns an $n \times g$ matrix of ICE predictions (one curve per row, one grid value per column);
(ii) `pdp(model, X, j, grid)` – returns the length-g PD vector by averaging the ICE matrix column-wise.

Solution

We implement both functions and test them on simulated data from b).

R

`ice()` repeats every observation once per grid value, sweeps the grid over column j and reshapes the vectorized model call into an $n \times g$ matrix; `pdp()` averages it column-wise.

```r
## f: prediction function | X: n*p data frame | j: column name or index | grid: vector of g values
ice = function(f, X, j, grid) {
  j = if (is.numeric(j)) names(X)[j] else j        # allow numeric or name
  g = length(grid)
  n = nrow(X)
  X_rep = X[rep(seq_len(n), each = g), ]           # repeat rows
  X_rep[[j]] = rep(grid, times = n)                # grid sweep on column j
  ice_mat = matrix(
    f(X_rep[[1]], X_rep[[2]]),                     # vectorised call
    nrow = n, ncol = g, byrow = TRUE
  )
  colnames(ice_mat) = grid
  ice_mat
}

pdp = function(f, X, j, grid) {
  colMeans(ice(f, X, j, grid))
}
```

Python

`ice()` repeats every observation once per grid value, sweeps the grid over column j and reshapes the vectorized model call into an $n \times g$ matrix; `pdp()` averages it column-wise.

```python
def ice(f, X, j, grid):
    """ICE matrix: one curve per row (observation), one column per grid value.

    f: prediction function | X: np.array (n, p) | j: feature index | grid: g values
    """
    curves = np.empty((X.shape[0], len(grid)))
    for g_id, val in enumerate(grid):
        X_rep = X.copy()
        X_rep[:, j] = val                          # grid sweep on column j
        curves[:, g_id] = f(X_rep[:, 0], X_rep[:, 1])  # vectorised call
    return curves


def pdp(f, X, j, grid):
    return ice(f, X, j, grid).mean(axis=0)
```

To check the functions, we simulate data according to b) and plot the ICE curves together with the PDP:

R

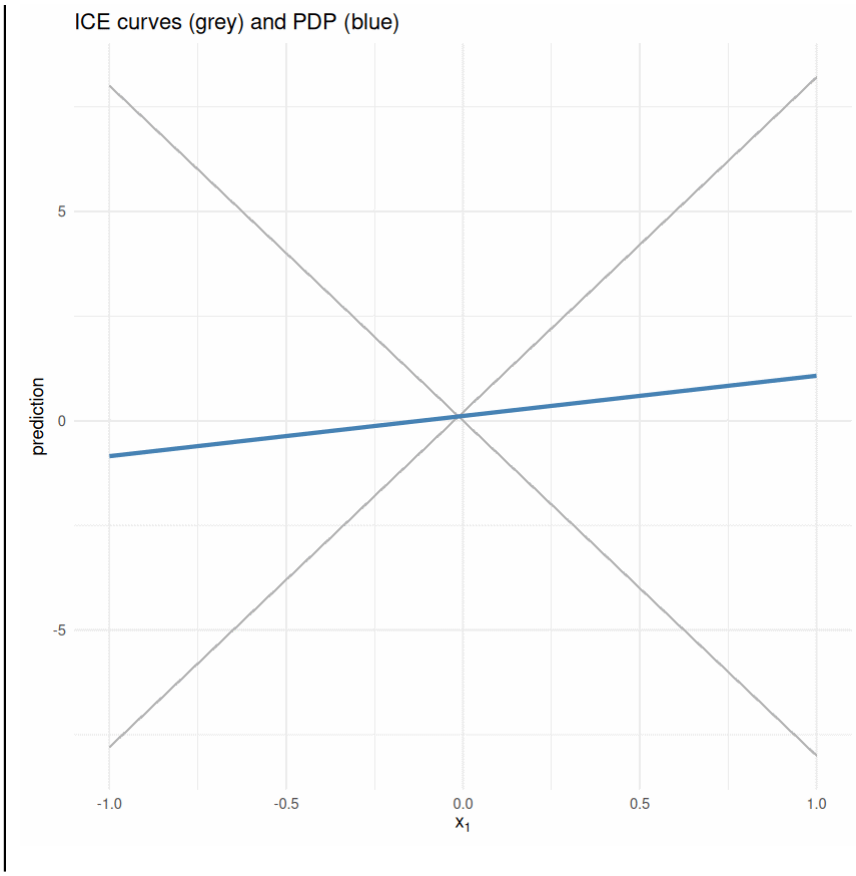
```r
## linear model with known coefficients (Exercise 1 b)
f_hat = function(x1, x2,
  beta0 = 0, beta1 = -8, beta2 = 0.2, beta3 = 16) {
  beta0 + beta1 * x1 + beta2 * x2 + beta3 * x1 * x2
}

set.seed(42)
n = 100
X = data.frame(
  x1 = runif(n, -1, 1),          # Unif(-1,1)
  x2 = rbinom(n, 1, 0.5)         # Bernoulli(0.5)
)
grid = seq(-1, 1, by = 0.25)     # evaluation grid for x1

ICE = ice(f_hat, X, "x1", grid)
PDP = pdp(f_hat, X, "x1", grid)

## reshape to long format for ggplot2
ice_long = data.frame(
  id   = rep(seq_len(nrow(ICE)), each = ncol(ICE)),
  x1   = rep(grid, times = nrow(ICE)),
  pred = as.vector(t(ICE))
)
pdp_df = data.frame(x1 = grid, pred = PDP)

ggplot() +
  geom_line(data = ice_long,
    aes(x = x1, y = pred, group = id),
    colour = "grey70", linewidth = 0.3) +
  geom_line(data = pdp_df,
    aes(x = x1, y = pred),
    colour = "steelblue", linewidth = 1.2) +
  labs(title = "ICE curves (grey) and PDP (blue)",
    x = expression(x[1]),
    y = "prediction") +
  theme_minimal()
```


ICE curves (grey) and PDP (blue)
prediction
5
0
-5
-1.0
-0.5
0.0
0.5
1.0
x_1

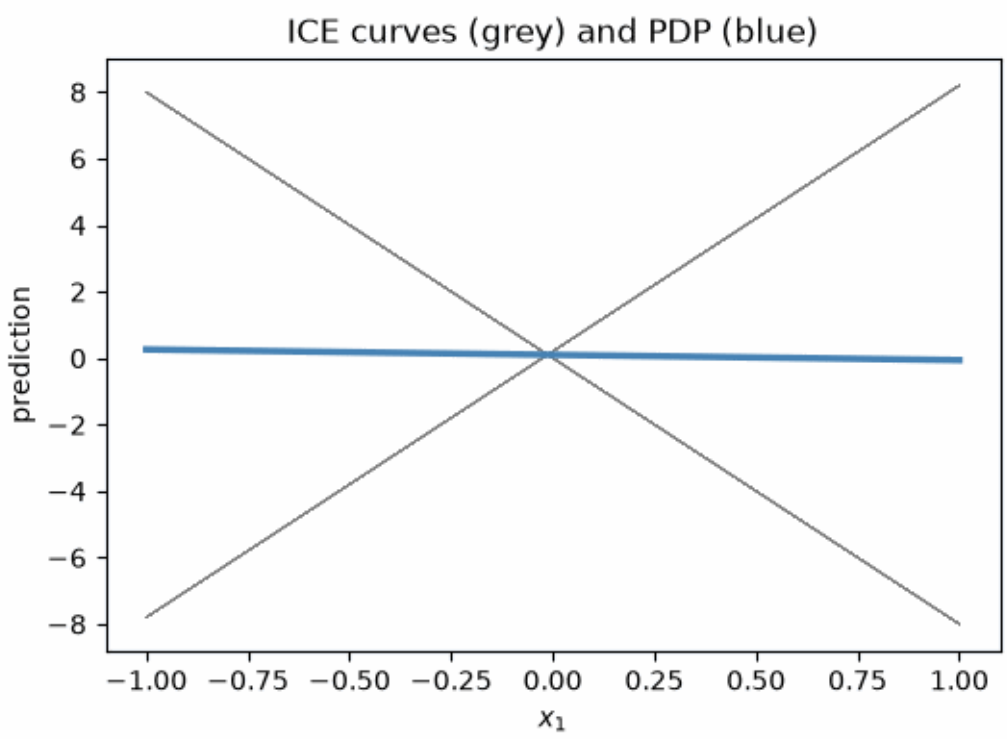
Python

```
# linear model with known coefficients (Exercise 1 b)
def f_hat(x1, x2, beta0=0, beta1=-8, beta2=0.2, beta3=16):
    return beta0 + beta1 * x1 + beta2 * x2 + beta3 * x1 * x2


rng = np.random.default_rng(42)
n = 100
X = np.column_stack([
    rng.uniform(-1, 1, n),       # Unif(-1,1)
    rng.binomial(1, 0.5, n),     # Bernoulli(0.5)
]).astype(float)
grid = np.arange(-1, 1.01, 0.25)  # evaluation grid for x1

ICE = ice(f_hat, X, 0, grid)
PDP = pdp(f_hat, X, 0, grid)

fig, ax = plt.subplots(figsize=(6, 4))
for curve in ICE:
    ax.plot(grid, curve, color="grey", linewidth=0.3, alpha=0.6)
ax.plot(grid, PDP, color="steelblue", linewidth=2.5)
ax.set_title("ICE curves (grey) and PDP (blue)")
ax.set_xlabel("$x_1$")
ax.set_ylabel("prediction")
plt.show()
```



As derived in b), the PDP is the constant function 0.1, while the two groups of ICE curves have slopes 8 and -8.

f) **Implementation: centered variants.** Extend the code with

(i) `c_ice(model, X, j, grid, ref = 1)` – centers every ICE curve by subtracting its value at the reference grid index `ref` (the "vertical shift" in the slides which is often set to the minimum value);

(ii) `c_pdp(model, X, j, grid, ref = 1)` – averages the centered ICE curves to obtain a centered PDP which is zero at the reference point.

Use the definitions of c-ICE and c-PDP from the lecture slides to guide your implementation.

Solution

Centering subtracts from each ICE curve its own prediction at the reference grid point; the centered PDP is the column mean of the centered ICE matrix and is zero at the reference point.

R

```r
## ref: index (1 ... g) of the reference grid point
c_ice = function(f, X, j, grid, ref = 1) {
  M = ice(f, X, j, grid)
  sweep(M, 1, M[, ref])         # subtract row-specific reference
}

c_pdp = function(f, X, j, grid, ref = 1) {
  colMeans(c_ice(f, X, j, grid, ref = ref))
}
```

Python

```python
def c_ice(f, X, j, grid, ref=0):
    """Centered ICE: subtract each curve's value at the reference grid index
    (0-based in Python)."""
    M = ice(f, X, j, grid)
    return M - M[:, [ref]]


def c_pdp(f, X, j, grid, ref=0):
    return c_ice(f, X, j, grid, ref=ref).mean(axis=0)
```

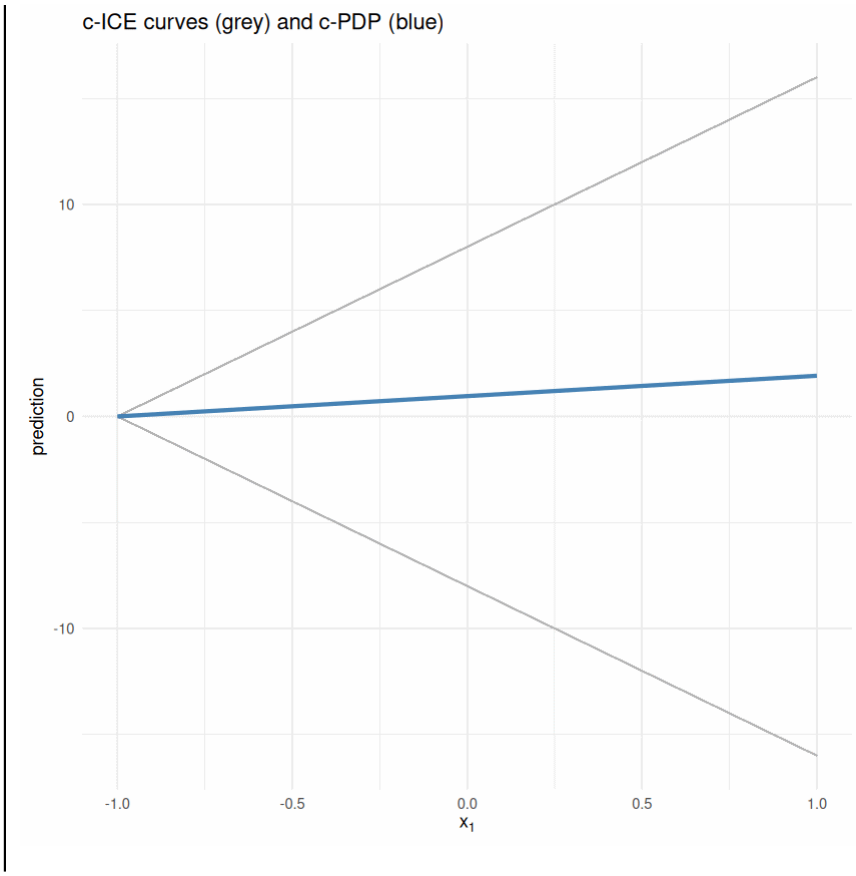
Plotting the centered curves makes the two interaction groups directly comparable at the reference point:

R

```r
cICE = c_ice(f_hat, X, "x1", grid, ref = which.min(grid))
cPDP = c_pdp(f_hat, X, "x1", grid, ref = which.min(grid))

cice_long = data.frame(
  id   = rep(seq_len(nrow(cICE)), each = ncol(cICE)),
  x1   = rep(grid, times = nrow(cICE)),
  pred = as.vector(t(cICE))
)
cpdp_df = data.frame(x1 = grid, pred = cPDP)

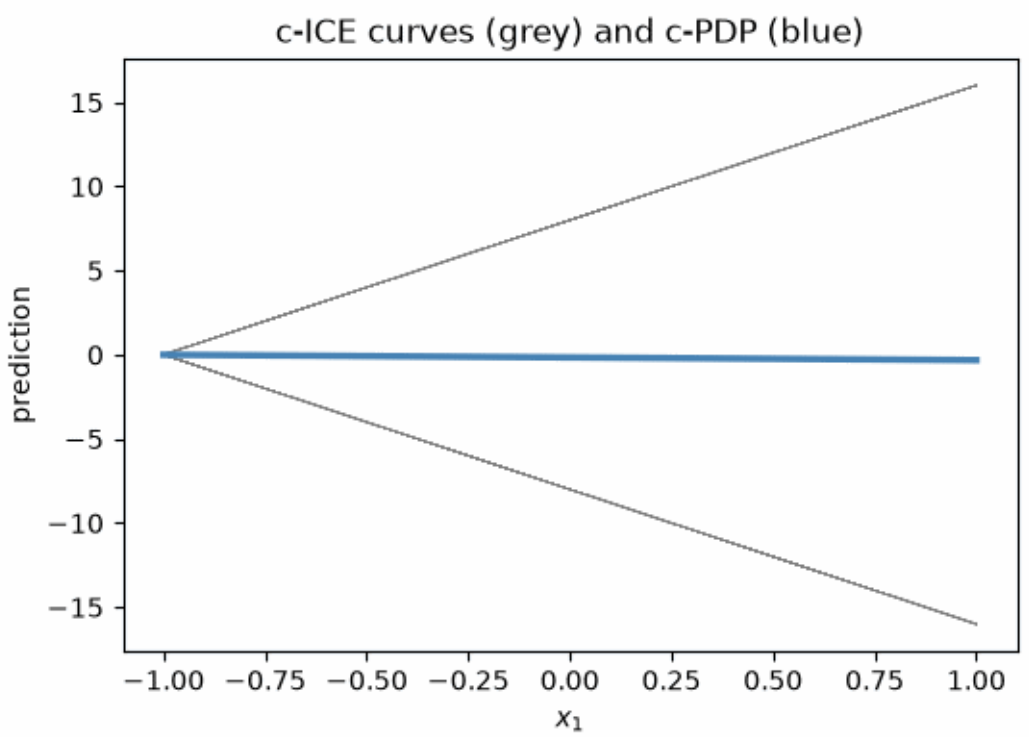
ggplot() +
  geom_line(data = cice_long,
    aes(x = x1, y = pred, group = id),
    colour = "grey70", linewidth = 0.3) +
  geom_line(data = cpdp_df,
    aes(x = x1, y = pred),
    colour = "steelblue", linewidth = 1.2) +
  labs(title = "c-ICE curves (grey) and c-PDP (blue)",
    x = expression(x[1]),
    y = "prediction") +
  theme_minimal()
```


c-ICE curves (grey) and c-PDP (blue)
prediction
10
0
-10
-1.0
-0.5
0.0
0.5
1.0
x_1

Python

```
ref = int(np.argmin(grid))
cICE = c_ice(f_hat, X, 0, grid, ref=ref)
cPDP = c_pdp(f_hat, X, 0, grid, ref=ref)

fig, ax = plt.subplots(figsize=(6, 4))
for curve in cICE:
    ax.plot(grid, curve, color="grey", linewidth=0.3, alpha=0.6)
ax.plot(grid, cPDP, color="steelblue", linewidth=2.5)
ax.set_title("c-ICE curves (grey) and c-PDP (blue)")
ax.set_xlabel("$x_1$")
ax.set_ylabel("prediction")
plt.show()
```



Exercise 2: Derivative, Forward, and Non-Linearity Effects

Learning goals

1. Derive derivative and forward marginal effects analytically.
2. Explain how curvature makes dME and fME differ for finite step sizes.
3. Compute and interpret the Non-Linearity Measure (NLM) as an R^2 of the local secant.

Let us consider the following non-linear regression model:

$$\hat{f}(x_1, x_2) = \theta_0 + \theta_1 x_1^2 + \theta_2 x_2^2 + \theta_{1,2} x_1 x_2$$

with parameters $\theta_0 = 0$, $\theta_1 = 1$, $\theta_2 = 0.5$, and $\theta_{1,2} = 2$.

a) Derive the analytic expression for the *derivative marginal effect* (dME) of feature x_1.

Solution

$$\text{dME}_1(x_1, x_2) = \frac{\partial \hat{f}}{\partial x_1} = 2\theta_1 x_1 + \theta_{1,2} x_2 = 2x_1 + 2x_2.$$

Discussion. Because the model is quadratic in x_1, the derivative is a *linear* function of both x_1 and x_2. If the surface were exactly linear, this rate would coincide with any finite difference.

b) Derive the expression for the *forward marginal effect* (fME) of x_1 with step size $h_1 > 0$.

Solution

$$\text{fME}_1(x_1, x_2; h_1) = \hat{f}(x_1 + h_1, x_2) - \hat{f}(x_1, x_2).$$

Expand $\hat{f}(x_1 + h_1, x_2)$:

$$\begin{aligned}\hat{f}(x_1 + h_1, x_2) &= \theta_1 (x_1 + h_1)^2 + \theta_2 x_2^2 + \theta_{1,2}(x_1 + h_1) x_2 \\ &= \theta_1 [x_1^2 + 2x_1 h_1 + h_1^2] + \theta_2 x_2^2 + \theta_{1,2} x_1 x_2 + \theta_{1,2} h_1 x_2.\end{aligned}$$

Subtracting $\hat{f}(x_1, x_2)$ gives

$$\boxed{\text{fME}_1(x_1, x_2; h_1) = 2\theta_1 x_1 h_1 + \theta_1 h_1^2 + \theta_{1,2} x_2 h_1} = 2x_1 h_1 + h_1^2 + 2x_2 h_1.$$

Discussion. The extra term $\theta_1 h_1^2$ captures curvature; it vanishes faster than linearly when $h_1 \to 0$, at which point fME converges to dME.

c) Let $x_1 = 1$, $x_2 = 2$, and $h_1 = 1$. Compute $\mathrm{dME}_1(1,2)$ and $\mathrm{fME}_1(1,2;h_1=1)$ and explain why these two results differ and interpret the additional terms appearing in the fME.

Solution

Numerical evaluation at $(x_1, x_2, h_1) = (1, 2, 1)$:

$$\begin{aligned}\mathrm{dME}_1(1,2) &= 2(1) + 2(2) = 6,\\ \mathrm{fME}_1(1,2;1) &= 2 \cdot 1 \cdot 1 + 1^2 + 2 \cdot 2 \cdot 1 = 2 + 1 + 4 = 7.\end{aligned}$$

Why different? The derivative 6 is the *instantaneous* slope. The finite step of 1 exposes curvature; the quadratic term makes the actual change go up to 7. Hence, using dME for a sizeable perturbation would misestimate (over- or under-predict) the effect.

d) **Non-Linearity Measure (NLM):** We want to test how well the secant (local linear approximation) fits the model output along the direction of x_1. The NLM defined below quantifies how well the local linear approximation (secant) explains the variation in the model output along the direction of h_1. For T grid points $(\mathbf{x}^{(i)})_{i=1}^{T}$, it is defined as:

$$\mathrm{NLM} = 1 - \frac{\overbrace{\sum_{i=1}^{T}\left[\hat{f}(\mathbf{x}^{(i)}) - g(t_i)\right]^2}^{\text{SSR: error of secant}}}{\underbrace{\sum_{i=1}^{T}\left[\hat{f}(\mathbf{x}^{(i)}) - \bar{f}\right]^2}_{\text{SST: total variance}}}.$$

- $t_i \in (0,1)$ parametrises the path (here only in x_1 direction): $\mathbf{x}^{(i)} = (x_1 + t_i h_1, x_2)$.
- $g(t_i) = \hat{f}(x_1, x_2) + t_i \cdot \mathrm{fME}_1(x_1, x_2; h_1)$ is the secant (straight line) through the two end-points.
- $\bar{f} = \frac{1}{T}\sum_{i=1}^{T} \hat{f}(\mathbf{x}^{(i)})$ is the mean prediction on the path.
- Hence NLM is an R^2: 1 means perfect linear fit, values $\ll 1$ reveal curvature.

Compute the NLM for $x_1 = 1$, $x_2 = 2$, and stepsize $h_1 = 0.5$. Interpret your result in relation to the fit of a local linear approximation using the $T = 9$ equidistant points $t_i = 0.1, 0.2, \ldots, 0.9$ below:

i	t_i	$x_1^{(i)} = 1 + 0.5t_i$	$\hat{f}(\mathbf{x}^{(i)})$	$g(t_i)$
1	0.1	1.05	7.3025	7.3250
2	0.2	1.10	7.6100	7.6500
3	0.3	1.15	7.9225	7.9750
4	0.4	1.20	8.2400	8.3000
5	0.5	1.25	8.5625	8.6250
6	0.6	1.30	8.8900	8.9500
7	0.7	1.35	9.2225	9.2750
8	0.8	1.40	9.5600	9.6000
9	0.9	1.45	9.9025	9.9250

Solution

Step 1: Path points. For the $T = 9$ equidistant points $t_i = 0.1, \ldots, 0.9$:

$$x_1^{(i)} = 1 + t_i h_1 = 1 + 0.5\, t_i, \qquad x_2^{(i)} = 2.$$

Step 2: Model and secant values.

$$f_i = \hat{f}(x_1^{(i)}, 2), \qquad g_i = \hat{f}(1,2) + t_i \cdot \mathrm{fME}_1(1,2;0.5), \qquad \hat{f}(1,2) = 7, \quad \mathrm{fME} = 3.25,$$

which are exactly the two last columns of the table.

Step 3: Compute the R^2.

$$\mathrm{NLM} = 1 - \frac{\sum_{i=1}^{9}(f_i - g_i)^2}{\sum_{i=1}^{9}(f_i - \bar{f})^2}, \qquad \bar{f} = \frac{1}{9}\sum_{i=1}^{9} f_i.$$

Numerically

$$\boxed{\mathrm{NLM} \approx 0.9967}.$$

Interpretation. Along the half-unit move in x_1, the quadratic surface is *almost* linear (NLM very close to 1). If we doubled h_1, the numerator would grow faster than the denominator, lowering NLM and signaling stronger curvature.

e) **Implementation:** Write three functions:

- A function `dME(f, x, j)` that computes a central difference approximation of the derivative marginal effect for feature x_j at input x.

- A function `fME(f, x, h)` that computes the forward marginal effect using a vector of stepsizes h.
- A function `NLM(f, x, h, t_vals)` that computes the Non-Linearity Measure as described in d) where `t_vals` are the grid points used to estimate the NLM.

Use them to confirm the results from sub-tasks c) and d).

Solution

The three functions follow the definitions directly; `dME()` uses a central difference with a small h:

R

```
## numerical derivative marginal effect (central difference)
dME = function(f, x, j, h = 1e-5) {
  x_forward = x
  x_backward = x
  x_forward[j] = x_forward[j] + h
  x_backward[j] = x_backward[j] - h
  (f(x_forward[1], x_forward[2]) - f(x_backward[1], x_backward[2])) / (2 * h)
}

## forward marginal effect with a vector of stepsizes
fME = function(f, x, h_vec) {
  x_perturbed = x + h_vec
  f(x_perturbed[1], x_perturbed[2]) - f(x[1], x[2])
}

## Non-Linearity Measure (R^2 of the secant along the path)
NLM = function(f, x, h_vec, t_vals = seq(0.1, 0.9, length.out = 9)) {
  x_path = sapply(t_vals, function(t) x + t * h_vec)
  f_vals = apply(x_path, 2, function(pt) f(pt[1], pt[2]))
  f_bar = mean(f_vals)
  fme_val = fME(f, x, h_vec)
  g_vals = sapply(t_vals, function(t) f(x[1], x[2]) + t * fme_val)
  1 - sum((f_vals - g_vals)^2) / sum((f_vals - f_bar)^2)
}
```

Python

```python
def dME(f, x, j, h=1e-5):
    """Central difference approximation of the derivative marginal effect."""
    x_forward = np.array(x, dtype=float)
    x_backward = np.array(x, dtype=float)
    x_forward[j] += h
    x_backward[j] -= h
    return (f(*x_forward) - f(*x_backward)) / (2 * h)


def fME(f, x, h_vec):
    """Forward marginal effect with a vector of stepsizes."""
    x = np.array(x, dtype=float)
    x_perturbed = x + np.array(h_vec, dtype=float)
    return f(*x_perturbed) - f(*x)


def NLM(f, x, h_vec, t_vals=np.linspace(0.1, 0.9, 9)):
    """Non-Linearity Measure (R^2 of the secant along the path)."""
    x = np.array(x, dtype=float)
    h_vec = np.array(h_vec, dtype=float)
    f_vals = np.array([f(*(x + t * h_vec)) for t in t_vals])
    f_bar = f_vals.mean()
    fme_val = fME(f, x, h_vec)
    g_vals = np.array([f(*x) + t * fme_val for t in t_vals])
    return 1 - np.sum((f_vals - g_vals) ** 2) / np.sum((f_vals - f_bar) ** 2)
```

Evaluating them for the model of this exercise confirms c) and d): $\mathrm{dME}_1(1,2) = 6$, $\mathrm{fME}_1(1,2;1) = 7$ and $\mathrm{NLM} \approx 0.9967$ for $h_1 = 0.5$.

R

```r
## model of Exercise 2
f_hat = function(x1, x2, theta0 = 0, theta1 = 1, theta2 = 0.5, theta12 = 2) {
  theta0 + theta1 * x1^2 + theta2 * x2^2 + theta12 * x1 * x2
}

x = c(1, 2)

cat("dME_1(1, 2)        =", dME(f_hat, x, j = 1), "\n")
cat("fME_1(1, 2; h = 1)  =", fME(f_hat, x, h_vec = c(1, 0)), "\n")
cat("fME_1(1, 2; h = 0.5)=", fME(f_hat, x, h_vec = c(0.5, 0)), "\n")
cat("NLM (h = 0.5)       =", NLM(f_hat, x, h_vec = c(0.5, 0)), "\n")
```

```
dME_1(1, 2)         = 6
fME_1(1, 2; h = 1)  = 7
fME_1(1, 2; h = 0.5)= 3.25
NLM (h = 0.5)       = 0.996714
```

Python

```python
# model of Exercise 2
def f_hat(x1, x2, theta0=0, theta1=1, theta2=0.5, theta12=2):
    return theta0 + theta1 * x1**2 + theta2 * x2**2 + theta12 * x1 * x2


x = (1, 2)

print("dME_1(1, 2)         =", dME(f_hat, x, j=0))
print("fME_1(1, 2; h = 1)  =", fME(f_hat, x, h_vec=[1, 0]))
print("fME_1(1, 2; h = 0.5)=", fME(f_hat, x, h_vec=[0.5, 0]))
print("NLM (h = 0.5)       =", NLM(f_hat, x, h_vec=[0.5, 0]))
```

```
dME_1(1, 2)         = 5.999999999994897
fME_1(1, 2; h = 1)  = 7.0
fME_1(1, 2; h = 0.5)= 3.25
NLM (h = 0.5)       = 0.9967140158610599
```

Visually, the tangent (slope dME) and the secant (slope fME/h_1) almost coincide with the prediction curve on this path, matching the NLM close to 1:

R

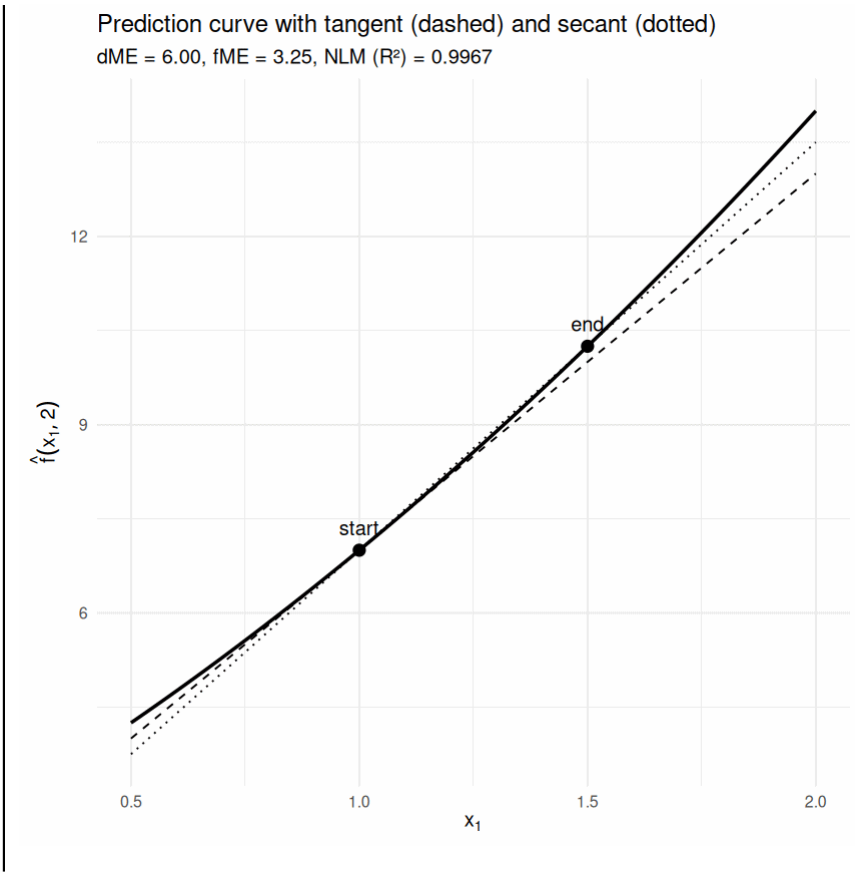
```r
h_vec = c(0.5, 0)       # step in x1 only
h1 = h_vec[1]
dme_val = dME(f_hat, x, j = 1)
fme_val = fME(f_hat, x, h_vec)
nlm_val = NLM(f_hat, x, h_vec)

## prediction curve in the neighbourhood of x1
x_seq = seq(x[1] - 0.5, x[1] + h1 + 0.5, length.out = 400)
curve_df = data.frame(x = x_seq, y = f_hat(x_seq, x[2]))

## tangent (dashed) and secant (dotted)
tan_df = data.frame(x = x_seq, y = f_hat(x[1], x[2]) + dme_val * (x_seq - x[1]))
sec_df = data.frame(x = x_seq, y = f_hat(x[1], x[2]) + fme_val / h1 * (x_seq - x[1]))

## start / end points of the step
pts_df = data.frame(
  x = c(x[1], x[1] + h1),
  y = c(f_hat(x[1], x[2]), f_hat(x[1] + h1, x[2])),
  lbl = c("start", "end")
)

ggplot(curve_df, aes(x, y)) +
  geom_line(linewidth = 1, colour = "black") +
  geom_line(data = tan_df, linetype = "dashed") +
  geom_line(data = sec_df, linetype = "dotted") +
  geom_point(data = pts_df, size = 3) +
  geom_text(data = pts_df, aes(label = lbl), vjust = -1) +
  labs(title = "Prediction curve with tangent (dashed) and secant (dotted)",
    subtitle = sprintf("dME = %.2f, fME = %.2f, NLM (R\u00b2) = %.4f",
      dme_val, fme_val, nlm_val),
    x = expression(x[1]), y = expression(hat(f)(x[1], 2))) +
  theme_minimal(base_size = 12)
```


Prediction curve with tangent (dashed) and secant (dotted)
dME = 6.00, fME = 3.25, NLM (R²) = 0.9967
12
9
6
$\hat{f}(x_1, 2)$
end
start
0.5
1.0
1.5
2.0
x_1

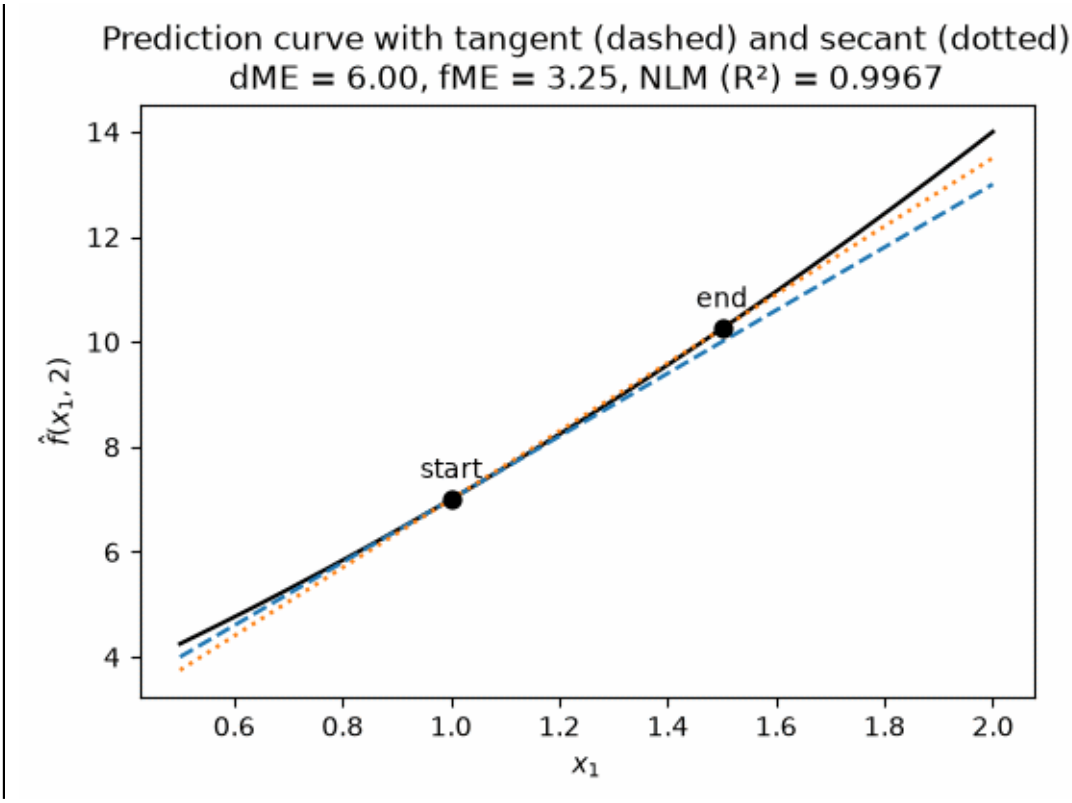
Python

```
h1 = 0.5
dme_val = dME(f_hat, x, j=0)
fme_val = fME(f_hat, x, h_vec=[h1, 0])
nlm_val = NLM(f_hat, x, h_vec=[h1, 0])

# prediction curve in the neighbourhood of x1
x_seq = np.linspace(x[0] - 0.5, x[0] + h1 + 0.5, 400)
y_curve = f_hat(x_seq, x[1])

# tangent (dashed) and secant (dotted)
y_tan = f_hat(*x) + dme_val * (x_seq - x[0])
y_sec = f_hat(*x) + fme_val / h1 * (x_seq - x[0])

fig, ax = plt.subplots(figsize=(6, 4))
ax.plot(x_seq, y_curve, color="black", linewidth=1.5)
ax.plot(x_seq, y_tan, linestyle="dashed", color="tab:blue")
ax.plot(x_seq, y_sec, linestyle="dotted", color="tab:orange")
pts_x = [x[0], x[0] + h1]
pts_y = [f_hat(*x), f_hat(x[0] + h1, x[1])]
ax.scatter(pts_x, pts_y, s=40, color="black", zorder=3)
for px, py, lbl in zip(pts_x, pts_y, ["start", "end"]):
    ax.annotate(lbl, (px, py), textcoords="offset points", xytext=(0, 8), ha="center")
ax.set_title("Prediction curve with tangent (dashed) and secant (dotted)\n"
             f"dME = {dme_val:.2f}, fME = {fme_val:.2f}, NLM (R\u00b2) = {nlm_val:.4f}")
ax.set_xlabel("$x_1$")
ax.set_ylabel("$\\hat{f}(x_1, 2)$")
plt.show()
```



Exercise 3: ALE

Learning goals

1. Explain how ALE uses intervals and local prediction differences to estimate feature effects.
2. Implement the ALE algorithm (interval bounds, accumulation, centering) from scratch.
3. Prepare and visualize ALE curves for a fitted model.

Accumulated local effects describe how features influence the prediction of a machine learning model on average. The following tasks guide you to implement ALE from scratch and to test your code on the wheat seeds dataset (data/wheat_seeds.csv). We prepare the data and fit a random forest; your functions are then tested on this model:

R

```r
## wheat seeds data: predict the (normalized) Type from the numeric features
df = na.omit(read.csv("../data/wheat_seeds.csv"))
df$Type = (df$Type - min(df$Type)) / (max(df$Type) - min(df$Type))
X = df[, setdiff(names(df), "Type")]

## fit a random forest on a 70% training split
set.seed(42)
train = sample(nrow(X), 0.7 * nrow(X))
model = randomForest(x = X[train, ], y = df$Type[train])
```

Python

```python
# wheat seeds data: predict the (normalized) Type from the numeric features
df = pd.read_csv("../data/wheat_seeds.csv").dropna()
y = (df["Type"] - df["Type"].min()) / (df["Type"].max() - df["Type"].min())
X = df.drop(columns="Type").to_numpy()
feature_names = list(df.drop(columns="Type").columns)

# fit a random forest on a 70% training split
rng = np.random.default_rng(42)
train = rng.choice(len(X), size=int(0.7 * len(X)), replace=False)
model = RandomForestRegressor(random_state=42).fit(X[train], y.to_numpy()[train])
```

a) Since ALE works with bins, we have to select bounds first. Usually the bins are chosen s.t. every bin has roughly the same number of data points. However, to make it more simple, we use the same length for every interval. Complete the function `get_bounds()` to select bounds between the minimum and maximum value of a feature by using `n_intervals`. Those bounds are used for the calculation of ALE in the next tasks.

Solution

Pseudocode of `get_bounds()`:

```
Require X: input data
Require s: index of the feature for calculating ALE
Require n_intervals: number of intervals
  x_s     <- s-th column of X
  x_s_min <- min value of x_s
  x_s_max <- max value of x_s
  return equidistant sequence of n_intervals + 1 grid points
         between x_s_min and x_s_max
```

R

```
get_bounds = function(X, s, n_intervals = 100) {
  ### Luckily for us the seq()-function already implements
  ### exactly what we want if we take two neighboring
  ### equidistant points as the boundaries of one interval.
  ### Now we need n_intervals + 1 points since we always
  ### need two points for one interval resulting
  ### in one point more than the number of intervals we want.
  seq(min(X[, s]), max(X[, s]), length.out = n_intervals + 1)
}

## bounds of 4 intervals for the first feature (Area)
get_bounds(X, s = 1, n_intervals = 4)
```

1. 10.59 2. 13.2375 3. 15.885 4. 18.5325 5. 21.18

Python

```
def get_bounds(X, s, n_intervals=100):
    """
    Calculates interval bounds given a X's min and max values.
    Only values in the s-th column are considered.

    Parameters:
        X (np.array with shape (num_instances, num_features)): Input data.
        s (int): Index of the feature x_s.
        n_intervals (int): Number of intervals.

    Returns:
        bounds (list): Values of bounds with `n_intervals`+1 entries.
    """
    x_s = X[:, s]
    x_s_min = np.min(x_s)
    x_s_max = np.max(x_s)
    bounds = list(np.linspace(x_s_min, x_s_max, n_intervals + 1))

    return bounds


# bounds of 4 intervals for the first feature (Area)
print(get_bounds(X, 0, n_intervals=4))
```

```
[np.float64(10.59), np.float64(13.2375), np.float64(15.885), np.float64(18.5325), np.flo
```

b) Implement the ALE algorithm from scratch inside the function `calculate_ale()` using the bounds from the previous task and the help of the lecture material. For each interval `k`:

- Select the observations inside the interval (if zero observations are found then return 0 for this interval). In general x_s can be counted as included if it is bigger than the lower bound and smaller or equal than the upper bound. However, make sure the elements in the first bound are included as well.
- Intervention: Use the relevant observations to create two new datasets:
  - `X_min`: Values at s-th feature position are replaced by the lower interval value z_{k-1}.
  - `X_max`: Values at s-th feature position are replaced by the higher interval value z_k.
- Prediction: Get the predictions for both `X_min` and `X_max`. Compute the differences in predictions of `X_max` to `X_min` and average them.

Use `cumsum` in R or `np.cumsum` in Python to perform the aggregation step and therefore to get accumulated values for each interval value. Finally, if the `center` parameter is set, center each interval by subtracting the average of all interval values.

Hint

`mapply` in R or `zip(bounds, bounds[1:])` in Python are nice tricks to iterate over the lower and upper bounds at the same time.

Solution

Pseudocode of `calculate_ale()`:

```
Require model: model with a predict method
Require X: input data
Require s: index of the feature for calculating ALE
Require n_intervals: number of intervals
Require centered: whether to return centered or uncentered ALE
  bounds     <- get_bounds(X, s, n_intervals)
  lowerbound <- lower interval bounds
  upperbound <- upper interval bounds
  result     <- {}
  for i1 in lowerbound & i2 in upperbound:
      idx <- ids of datapoints of X in (i1, i2]   # first interval: [i1, i2]
      if idx is empty:
          diff <- 0
      else:
          X_min <- X[idx, ] with feature values of s replaced by i1
```

```
        X_max <- X[idx, ] with feature values of s replaced by i2
        y_min <- model predictions of X_min
        y_max <- model predictions of X_max
        diff  <- y_max - y_min
    result <- {result, mean of diff}
uncentered_ale <- cumulative sum of result
if centered:
    centered_ale <- uncentered_ale - (mean of uncentered_ale)
return bounds and either uncentered_ale or centered_ale
```

R

```r
calculate_ale = function(model, X, s, n_intervals = 100, centered = FALSE) {
  ### Get the feature column specified by s from the input data.
  x_s = X[, s]

  ### Save all possible lower and upper bounds in
  ### lowerbounds and upperbounds, respectively.
  boundary_points = get_bounds(X, s, n_intervals)
  lowerbounds = boundary_points[1:n_intervals]
  upperbounds = boundary_points[2:(n_intervals + 1)]

  ### Now we want to iterate over all intervals. For this we
  ### will use mapply() in the following but for readability we define the
  ### function to apply before. This function returns the
  ### uncentered ALE value for one interval given the left and the right
  ### boundary of the interval.
  uncentered_ale_calculation = function(left_boundary, right_boundary) {
    ### The first interval is of the form [left_boundary, right_boundary]
    ### (so that we do not ignore the points with the minimal feature value).
    if (left_boundary == lowerbounds[1]) {
      idxs = which(x_s >= left_boundary & x_s <= right_boundary)
    }
    ### All other intervals are of the form (left_boundary, right_boundary]
    ### (so that no point falls into two intervals).
    else {
      idxs = which(x_s > left_boundary & x_s <= right_boundary)
    }

    ### If there is no point in the interval return 0
    ### (as stated in the task).
    if (length(idxs) == 0) {
      return(0)
    }

    ### Else we compute the ALE value. The first step is to
    ### replace the feature values of all the points that fall into the interval
    ### with the left and the right boundary of the interval.
    X_min = X[idxs, ]
    X_max = X[idxs, ]
    X_min = replace(X_min, s, left_boundary)
    X_max = replace(X_max, s, right_boundary)

    ### Then we let the model predict these "new" data points.
    y_min = predict(model, X_min)
    y_max = predict(model, X_max)

    ### We return the arithmetic mean of the prediction differences.
    mean(y_max - y_min)
  }

  ### Now we can calculate the ALE values all at once with the mapply()-function.
  ### We accumulate them via the cumsum()-function.
  ale_values = cumsum(mapply(uncentered_ale_calculation, lowerbounds, upperbounds))

  ### If the centered argument is set to TRUE the user wants us to center the ALE values.
  if (centered)
    ale_values = ale_values - mean(ale_values)

  ### Return the boundary points as well as the centered or uncentered ALE values.
  list(bounds = boundary_points, ale = ale_values)
}

## uncentered ALE of the first feature with 4 intervals
calculate_ale(model, X, s = 1, n_intervals = 4)
```

$bounds 1. 10.59 2. 13.2375 3. 15.885 4. 18.5325 5. 21.18

$ale 1. 0.142390888888889 2. 0.220174530864197 3. 0.235019843364198 4. 0.235019843364198

Python

```python
def calculate_ale(model, X, s, n_intervals=100, centered=False):
    """
    Compute the accumulated local effect of a numeric continuous feature.

    Parameters:
        model: Model which has a predict method.
        X (np.array with shape (num_instances, num_features)): Input data.
        s (int): Index of the feature x_s.
        n_intervals (int): How many intervals should be used.
        centered (bool): Whether to return uncentered or centered ALE.

    Returns:
        bounds (list): See function `get_bounds`.
        ale (list with `n_intervals` entries): Values of ALE.
    """
    x_s = X[:, s]
    bounds = get_bounds(X, s, n_intervals)

    uncentered_ale = []

    # i1 = z_{k-1}
    # i2 = z_k
    for i1, i2 in zip(bounds, bounds[1:]):
        # Get ids in between the interval
        if i1 == bounds[0]:
            # Make sure to include the point on the first interval too
            idx = np.where((x_s >= i1) & (x_s <= i2))[0]
        else:
            idx = np.where((x_s > i1) & (x_s <= i2))[0]

        if len(idx) == 0:
            uncentered_ale.append(0)
            continue

        # Now we replace the values at position s one time with the smallest value
        # and one time with the highest value in this interval.
        X_min = X[idx, :]
        X_max = X[idx, :]
        X_min[:, s] = i1
        X_max[:, s] = i2

        # And get the new predictions
        y_min = model.predict(X_min)
        y_max = model.predict(X_max)

        # Calculate difference now
        diff = y_max - y_min
        uncentered_ale_k = np.sum(diff) / len(idx)
        uncentered_ale.append(uncentered_ale_k)

    # Now accumulate
    uncentered_ale = np.cumsum(uncentered_ale)

    if centered:
        # Center ALE here
        mean_uncentered_ale = np.sum(uncentered_ale) / len(uncentered_ale)
        centered_ale = [ale_k - mean_uncentered_ale for ale_k in uncentered_ale]

        return bounds, centered_ale
    else:
        return bounds, uncentered_ale


# uncentered ALE of the first feature with 4 intervals
bounds, ale = calculate_ale(model, X, 0, n_intervals=4)
```

```
bounds: [10.59  13.238 15.885 18.532 21.18 ]
ALE:    [0.0112 0.1463 0.1647 0.1647]
```

c) The function `calculate_ale()` returns both the used bounds and the ALE data. To use those data with `ggplot2` (R) or `matplotlib` (Python), you have to get the corresponding interval centers. Implement the function `prepare_ale()`, which should call `calculate_ale()` first and then compute the averages/centers per interval. In the end `prepare_ale()` should return the centers plus the corresponding ALE values – either as a `data.frame` (R) or two vectors (Python).

Solution

Pseudocode of `prepare_ale()`:

```
Require model, X, s, n_intervals, centered (as before)
  bounds, y  <- calculate_ale(X, s, n_intervals, centered)
  lowerbound <- lower interval bounds
  upperbound <- upper interval bounds
  x          <- center of each interval (middle of lowerbound and upperbound)
  return x and y
```

R

```r
prepare_ale = function(model, X, s, n_intervals = 100, centered = TRUE) {
  ### Get the ALE values via our function from task b).
  ale = calculate_ale(model, X, s, n_intervals, centered)

  ### Get the bounds and y from the list.
  bounds = unlist(ale[1])
  y = unlist(ale[2])

  ### Reconstruct the lowerbounds and upperbounds vectors.
  lowerbounds = bounds[1:n_intervals]
  upperbounds = bounds[2:(n_intervals + 1)]

  ### Get the center of each interval.
  x = rowMeans(cbind(lowerbounds, upperbounds))

  ### Now for ggplot2 the expected format is a data.frame.
  data.frame(x = x, y = y)
}
```

Python

```
def prepare_ale(model, X, s, n_intervals=100, centered=False):
    """
    Uses `calculate_ale` to prepare x and y data, which can be used
    by matplotlib directly.

    Parameters:
        model: Model which has a predict method.
        X (np.array with shape (num_instances, num_features)): Input data.
        s (int): Index of the feature x_s.
        n_intervals (int): How many intervals should be used.
        centered (bool): Whether to return uncentered or centered ALE.

    Returns:
        x (list or 1D np.ndarray): Centers of two bounds. `n_intervals` entries.
        y (list or 1D np.ndarray): ALE values. `n_intervals` entries.
    """
    bounds, y = calculate_ale(model, X, s, n_intervals, centered)
    x = [(i1 + i2) / 2 for i1, i2 in zip(bounds, bounds[1:])]

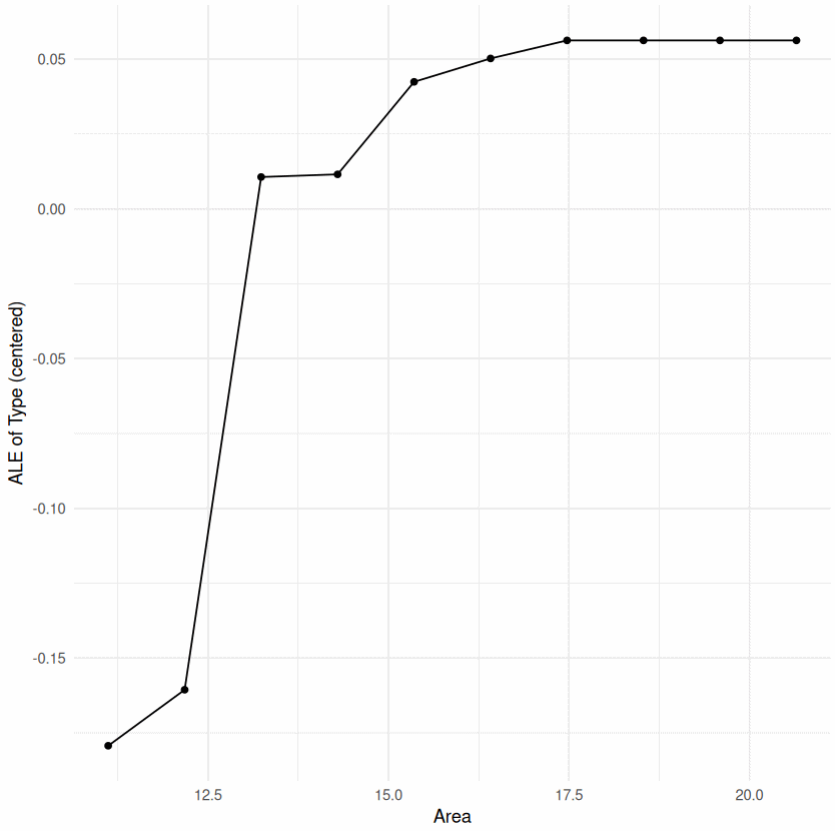
    return x, y
```

Plotting the prepared values shows the (centered) ALE curve of the first feature:

R

```
prepared_ale = prepare_ale(model, X, s = 1, n_intervals = 10, centered = TRUE)

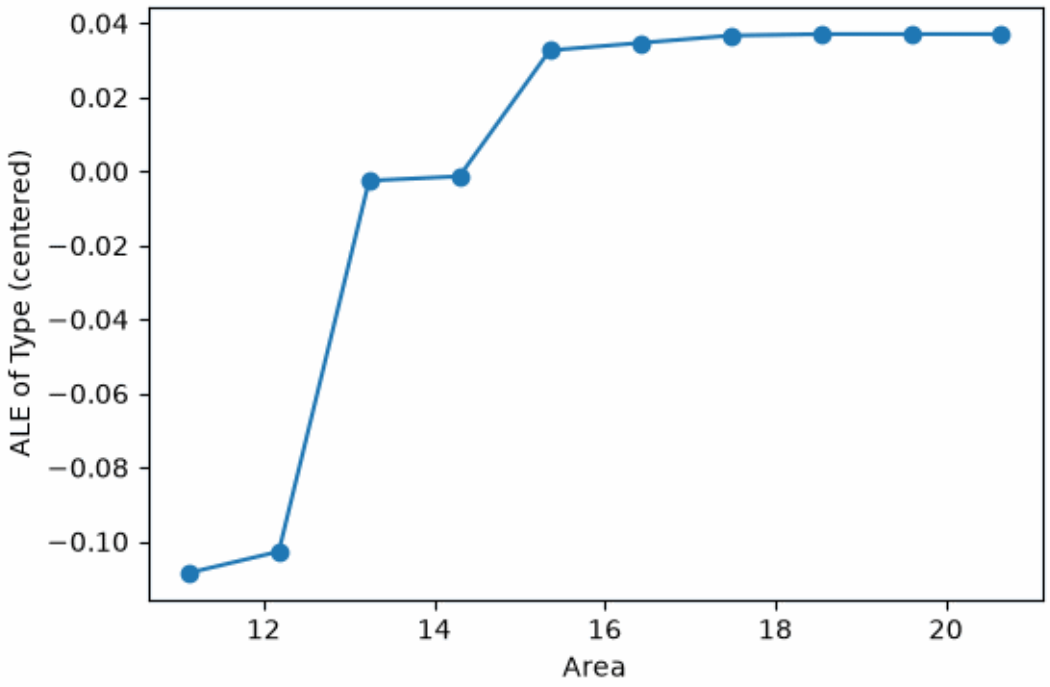
ggplot(data = prepared_ale, mapping = aes(x = x, y = y)) +
  geom_line() +
  geom_point() +
  labs(x = "Area", y = "ALE of Type (centered)") +
  theme_minimal()
```



Python

```
x_ale, y_ale = prepare_ale(model, X, 0, n_intervals=10, centered=True)

fig, ax = plt.subplots(figsize=(6, 4))
ax.plot(x_ale, y_ale, marker="o")
ax.set_xlabel("Area")
ax.set_ylabel("ALE of Type (centered)")
plt.show()
```



Exercise 4: Categorical ALE

Learning goals

1. Interpret PDPs and ALE plots for categorical features in a risk-modelling context.
2. Explain when ALE plots are preferable to PDPs.
3. Implement the distributional distance for categorical features used to order the levels of a nominal feature.

In this exercise, we use the German Credit dataset (data/credit.csv) and a random forest to predict whether a customer is a high or low risk for a bank. An overview of the dataset is given below:

	name	type	mean	nlevs
1	credit_risk	factor		2
2	age	integer	35.55	0
3	amount	integer	3271.26	0
4	credit_history	factor		5
5	duration	integer	20.90	0
6	employment_duration	factor		5
7	personal_status_sex	factor		4
8	purpose	factor		10

Since the decision of whether a person gets a loan can have serious implications on the person's life, banks are subordinate to regulations and must disclose the underlying mechanism of their used model to authorities. Since looking on the single trees is not feasible to uncover the internals of a random forest, (model-agnostic) interpretation methods should help to unfold the underlying mechanisms and to explain specific decisions.

a) Since the regulations require that the model does not discriminate against certain demographic groups, you want to evaluate the feature effect of `personal_status_sex` of the fitted forest model. Your colleague provides the following PDP. Interpret the PDP.

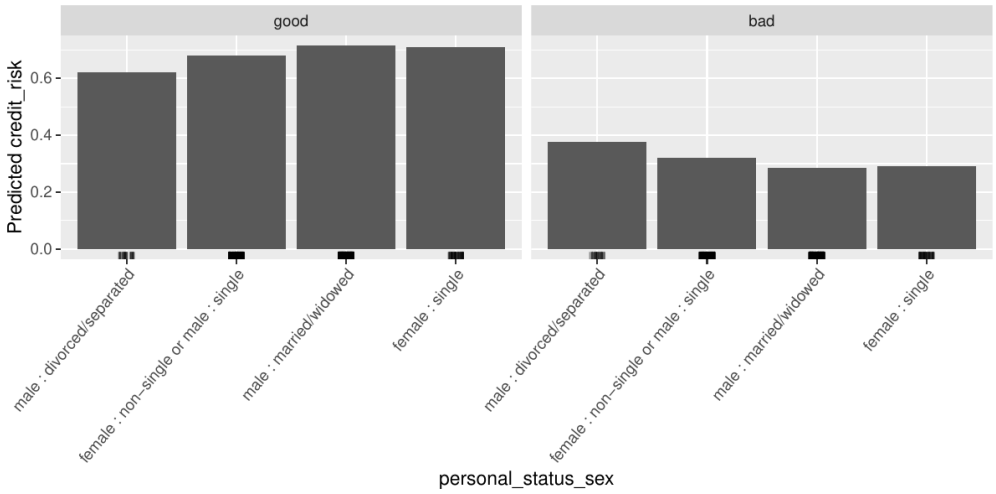


Figure 1: PDP of personal_status_sex

Solution

Overall, all customers, regardless of their personal status and gender, have on average a high probability of being a low (good) risk for the bank. The average marginal prediction for divorced or separated male customers reveals a slightly higher risk for this group.

b) You also request an ALE plot from your colleague. Explain: Why and in which situations is inspecting ALE plots besides PDPs advisable?

Solution

The ALE is faster to compute and unbiased. Unbiasedness means that it does not suffer from the extrapolation problem which is especially apparent in PDPs when features are correlated.

c) Because the ALE model accumulates effects in a specific direction, the feature values must have an order by definition. However, there is no natural order to categorical/nominal features like `personal_status_sex`. In order to derive an artificial order, Molnar 2022 (Chapter 8.2) proposes to order the categories of the feature x_s according to their similarity based on other/remaining features $x_c \in X_C$. Read the corresponding subsection in Molnar 2022.

In detail, the approach has the following steps:

(i) For each feature $x_c \in X_C$ do the following:

- Get the conditional empirical distributions of x_c given the categories of x_s: For numeric feature x_c, estimate the cumulative distribution function for each category of x_s separately and derive its values at predefined points (e.g., at

deciles of x_c). For categorical feature x_c derive the relative frequency tables for each category of x_s separately.
- Compute the pairwise distances of distributions for each pair of categories of x_s: For numeric feature x_c, the distance is equal to the absolute maximum point-wise distance of the two empirical distribution functions. For categorical feature x_c, the distance is equal to the sum of the absolute difference of both relative frequency tables.

(ii) Sum up the pairwise distances of the distributions over features x_c in X_C for each class pair of x_s.

(iii) Reduce the resulting distance matrix to a single dimension using multi-dimensional scaling.

(iv) Order the categories of x_s according to the obtained similarity values.

Your colleague already implemented some parts of the algorithm, namely `order_levels()` and the distance of distributions for a **numeric** feature x_c in `get_diff_numeric()`:

R

```r
credit = read.csv("../data/credit.csv", stringsAsFactors = TRUE)

order_levels = function(data, feature.name) {
  feature = data[, feature.name]
  others = setdiff(colnames(data), feature.name)
  feature.lev = unique(feature)

  ### Iterate over other features
  dists = lapply(others, function(k) {
    feature.k = data[, k]
    if (inherits(feature.k, "factor") | inherits(feature.k, "character")) {
      dists = get_diff_cat(feature.k, feature)
    } else {
      dists = get_diff_numeric(feature.k, feature)
    }
    dists
  })
  dists.cumulated.long = as.data.frame(Reduce(function(d1, d2) {
    d1$dist = d1$dist + d2$dist
    d1
  }, dists))

  ### Create a matrix of distances
  dists.cumulated = reshape2::dcast(dists.cumulated.long,
    class1 ~ class2, value.var = "dist")[, -1]
  rownames(dists.cumulated) = colnames(dists.cumulated)

  ### conduct multi-dimensional scaling (here: principal coordinates analysis)
  ### based on dissimilarity matrix it assigns
  ### to each item a location in a low dimensional space
  ### the closer the location, the more similar the items are.
  scaled = cmdscale(dists.cumulated, k = 1)
  feature.lev[order(scaled)]
}

get_diff_numeric = function(feature.k, feature.j) {
  ### set up data.frame which we will fill later
  dists = expand.grid(unique(feature.j), unique(feature.j))
  colnames(dists) = c("class1", "class2")

  ### get decentiles
  quants = quantile(feature.k, probs = seq(0, 1, length.out = 10),
    na.rm = TRUE, names = FALSE)

  ### derive empirical distribution function for each category
  ecdfs = data.frame(lapply(unique(feature.j), function(lev) {
    x.ecdf = ecdf(feature.k[feature.j == lev])(quants)
    return(x.ecdf)
  }))
  colnames(ecdfs) = unique(feature.j)

  ### get pairwise absolute distances of empirical distribution function
  ### between the different categories
  ecdf.dists.all = abs(ecdfs[, dists$class1] - ecdfs[, dists$class2])

  ### get maximum distance over decentiles for each pair of categories
  dists$dist = apply(ecdf.dists.all, 2, max)
  dists
}
```

Python

```
credit = pd.read_csv("../data/credit.csv")


def order_levels(data, feature_name):
    """
    Orders levels of a nominal scaled feature according to other features in a dataset.

    Parameters:
        data (pd.DataFrame): Input data.
        feature_name (str): Name of the categorical feature which should be ordered.

    Returns: np.array (str) of ordered class labels
    """
    feature = data[feature_name].values
    others = list(data.columns)
    others.remove(feature_name)
    feature_lev = np.unique(feature)

    dists = []

    # Iterate over other features, sum up distances
    for i in range(len(others)):
        k = others[i]
        feature_k = data[k].values
        if (feature_k.dtype.kind != 'i') & (feature_k.dtype.kind != 'f'):
            ds = get_diff_cat(feature_k, feature)
        else:
            ds = get_diff_numeric(feature_k, feature)

        if i == 0:
            dists = ds
        else:
            dists['dist'] = dists['dist'] + ds['dist']

    # Create a matrix of distances
    dists = dists.pivot(index='class1', columns='class2', values='dist')

    # conduct classical multi-dimensional scaling (principal coordinates
    # analysis, like R's cmdscale): double-center the squared distance
    # matrix and project on the first principal coordinate
    D2 = np.asarray(dists, dtype=float) ** 2
    J = np.eye(len(D2)) - np.ones_like(D2) / len(D2)
    B = -0.5 * J @ D2 @ J
    eigval, eigvec = np.linalg.eigh(B)
    scaled = eigvec[:, -1] * np.sqrt(max(eigval[-1], 0))

    order = feature_lev[scaled.argsort()]

    return order


def get_diff_numeric(feature_k, feature_j):
    """
    Calculates the pairwise distances of classes of j of a numeric feature k.

    Parameters:
        feature_k (np.array): Values of the numeric feature.
        feature_j (np.array): Values of the categorical feature for which
            similarity based on feature k should be assessed.

    Returns: pd.DataFrame with three columns:
        class1: name of the first class
        class2: name of the second class
        dist: the distributional distance between the two classes for feature k
    """
```

Your colleague needs your help with implementing the computation of the distance of distribution for a **categorical** feature x_c in `get_diff_cat()`:

Hint

In R, the command `expand.grid(levels(feature.j), levels(feature.j))` creates the first two columns in your resulting dataset – the pairwise class combinations for x_s.

- Have a look on the functions `order_levels()` and `get_diff_numeric()`. Locate the steps of above's algorithmic description in the underlying code.
- Implement the missing rows in `get_diff_cat()`.
- Test your function with `personal_status_sex` as x_s and `employment_duration` as x_c.

Your returned result should look similar to this:

class1	class2	dist
male : married/widowed	male : married/widowed	0.00
female : non-single or male : single	male : married/widowed	0.22
male : divorced/separated	male : married/widowed	0.19
...	...	...

Solution

`get_diff_cat()` computes the pairwise sum of the absolute differences of relative frequencies in the categories of a categorical feature: the relative cross table of x_s and x_c is computed (per class of x_s, the relative frequencies of the classes of x_c sum up to 1), and for each pair of classes of x_s the absolute differences of the two frequency rows are summed up.

R

```
get_diff_cat = function(feature.k, feature.j) {
  ### set up data.frame which we will fill later
  dists = expand.grid(unique(feature.j), unique(feature.j))
  colnames(dists) = c("class1", "class2")

  ### get relative frequency table
  x.count = as.numeric(table(feature.j))
  A = table(feature.j, feature.k) / x.count

  ### compute pairwise absolute distances
  dists$dist = rowSums(
    abs(A[as.character(dists[, "class1"]), ] - A[as.character(dists[, "class2"]), ]))
  dists
}
```

Python

```python
def get_diff_cat(feature_k, feature_j):
    """
    Calculates the pairwise distances of classes of j of a categorical feature k.

    Parameters:
        feature_k (np.array): Values of the categorical feature for which
            relative frequencies per class are calculated.
        feature_j (np.array): Values of the categorical feature for which
            similarity based on feature k should be assessed.

    Returns: pd.DataFrame with three columns:
        class1: name of the first class
        class2: name of the second class
        dist: the distributional distance between the two classes for feature k
    """
    # set up data.frame which we will fill later
    feature_lev = np.unique(feature_j)
    param_grid = {'class1': feature_lev, 'class2': feature_lev}
    dists = pd.DataFrame(ParameterGrid(param_grid))

    # get relative frequency table
    xcount = np.unique(feature_j, return_counts=True)[1]
    A = pd.crosstab(feature_k, feature_j) / xcount

    # compute pairwise absolute distances
    ecdf_dists = []
    for i1, i2 in zip(dists.class1.values, dists.class2.values):
        ecdf_dists.append(sum(abs(A[i1] - A[i2])))
    dists["dist"] = ecdf_dists

    return dists
```

For our task at hand, we obtain the following distances:

R

```
get_diff_cat(feature.k = credit[, "employment_duration"],
  feature.j = credit[, "personal_status_sex"])
```

A data.frame: 16 × 3

class1 <fct>	class2 <fct>	dist <dbl>
male : married/widowed	male : married/widowed	0.00000
female : non-single or male : single	male : married/widowed	0.44829
male : divorced/separated	male : married/widowed	0.37474
female : single	male : married/widowed	0.49761
male : married/widowed	female : non-single or male : single	0.44829
female : non-single or male : single	female : non-single or male : single	0.00000
male : divorced/separated	female : non-single or male : single	0.28645
female : single	female : non-single or male : single	0.15862
male : married/widowed	male : divorced/separated	0.37474
female : non-single or male : single	male : divorced/separated	0.28645
male : divorced/separated	male : divorced/separated	0.00000
female : single	male : divorced/separated	0.29217
male : married/widowed	female : single	0.49761
female : non-single or male : single	female : single	0.15862
male : divorced/separated	female : single	0.29217
female : single	female : single	0.00000

Python

```
get_diff_cat(feature_k=credit["employment_duration"].values,
             feature_j=credit["personal_status_sex"].values)
```

	class1	class2	dist
0	female : non-single or male : single	female : non-single or male : single	0.000000
1	female : non-single or male : single	female : single	0.158626
2	female : non-single or male : single	male : divorced/separated	0.286452
3	female : non-single or male : single	male : married/widowed	0.448293
4	female : single	female : non-single or male : single	0.158626
5	female : single	female : single	0.000000
6	female : single	male : divorced/separated	0.292174
7	female : single	male : married/widowed	0.497620

8	male : divorced/separated	female : non-single or male : single	0.286452
9	male : divorced/separated	female : single	0.292174
10	male : divorced/separated	male : divorced/separated	0.000000
11	male : divorced/separated	male : married/widowed	0.374745
12	male : married/widowed	female : non-single or male : single	0.448293
13	male : married/widowed	female : single	0.497620
14	male : married/widowed	male : divorced/separated	0.374745
15	male : married/widowed	male : married/widowed	0.000000

d) How did your function order the levels of `personal_status_sex`? Does this order make sense to you?

Solution

Running `order_levels()` on the full dataset gives the following ordering of `personal_status_sex`:

R

```
order_levels(credit, "personal_status_sex")
```

1. female : single 2. female : non-single or male : single 3. male : married/widowed 4. male : divorced/separated

Levels: 1. 'female : non-single or male : single' 2. 'female : single' 3. 'male : divorced/separated' 4. 'male : married/widowed'

Python

```
order_levels(credit, "personal_status_sex")
```

```
array(['male : divorced/separated', 'male : married/widowed',
       'female : non-single or male : single', 'female : single'],
      dtype=object)
```

The ordering seems to be feasible, since categories including females are close to each other and also categories with males. Also the ordering of males according to their relationship status seems to make sense, since typically the process is: single, then married and then divorced :-).

e) **Bonus:** Critically assess whether PDP/ALE alone replace a complete fairness audit.

Solution

ALE and PDP are global interpretation tools which base their insights on averages (of predictions or prediction differences) over whole test sets. Indeed vulnerable groups are typically not the majority of a population but have a low proportion, and biases might be overlooked. Therefore, local explanation tools should be consulted, in addition to these methods in order to identify pointwise biases or discriminatory behavior.